Fundamentals and Applications of Artificial Intelligence

AI Fundamentals Project Portfolio

Search, Learning, Decision Making, Games, Logic, and LLM-Symbolic Reasoning

Team Members: Mohammad Saleh Mahdinejad
Fatemeh Sayadzadeh

Repository: `AI_Fundamentals`

Document Type: Consolidated technical report

June 5, 2026

Abstract

This report documents a consolidated portfolio of five artificial intelligence course projects. The projects begin with classical graph search and optimization, continue through regression, local search, Markov decision processes, reinforcement learning, and adversarial games, and end with first-order logic and hybrid LLM-symbolic reasoning. The final repository was built by reviewing the original assignment repositories, preserving the executable code and required assets, removing classroom metadata and temporary artifacts, cleaning sensitive configuration, and writing a single professional documentation layer around the complete body of work.

Contents

1 Introduction

The purpose of this repository is to present the full output of the AI fundamentals course as one coherent portfolio rather than as five isolated classroom repositories. Each original repository focused on a different family of AI methods:

- deterministic and informed search,
- numerical learning and gradient-based optimization,
- stochastic decision making and reinforcement learning,
- adversarial search for competitive games,
- symbolic inference using first-order logic,
- and hybrid reasoning where an LLM generates constraints validated by a symbolic solver.

The final repository keeps the phases independent enough to run separately, but it also creates one shared README, one shared dependency file, and this report as the main explanation of the implementation.

1.1 Source Material Reviewed

The consolidated report is based on both the code and the phase documents that were part of the original repositories. The following table summarizes the reviewed material.

Original Repository	Reviewed Documents	Main Code/Notebook Artifacts
`search-regression-anneal-star`	`phase1-doc.pdf`, `Report.pdf`	BFS/UCS/DLS/A* scripts, regression notebook, DNA center notebook
`mdp-unknown-star`	`phase2-doc.pdf`, `phase2-env-instruction.pdf`	MDP solver, Q-learning solver, DQN bonus implementation
`game-star`	`phase3.pdf`	PaintBattle environment, minimax agent, maps, binary enemy agents
`fol-star`	`phase4.pdf`	Minesweeper environment, PyDatalog solver, manual controller
`fol-llm-star`	`phase4-bonus.pdf`	Mastermind environment, LLM-Prolog solver, manual game

1.2 Final Repository Structure

```
AI_Fundamentals/
  README.md
  requirements.txt
  docs/
    AI_Fundamentals_Report.tex
    AI_Fundamentals_Report.pdf
    figures/
  projects/
```

```
phase-01-search-regression-annealing/
phase-02-mdp-reinforcement-learning/
phase-03-adversarial-search-game/
phase-04-fol-minesweeper/
phase-05-llm-fol-mastermind/
```

2 Phase 1: Search, Regression, and Local Search

2.1 Scope

Phase 1 contains three independent tasks. The first task is a grid-world search problem based on an Angry Birds style environment. The second task predicts asteroid diameter with a linear regression model trained by stochastic gradient descent implemented from scratch. The third task solves the closest DNA string problem using local search algorithms.

2.2 Part A: Grid Search

2.2.1 Problem Definition

The search environment is implemented in `search_algorithms/env/env.py`. The agent moves through a grid containing grass, bushes, rocks, targets, and an enemy following a periodic path. The objective is to collect all targets while avoiding collisions and minimizing unnecessary exploration. The environment exposes the search interface through methods such as:

- `get_successors()` for valid next states and costs,
- `is_goal_state()` for target completion,
- `is_collision_state()` for enemy and obstacle collisions,
- `get_enemy_path()` and `get_enemy_position()` for dynamic enemy reasoning,
- `get_targets_positions()` for remaining goals.

The environment uses non-uniform costs: grass is cheap, bushes are expensive, rocks block movement, target collection increases score, and enemy collision is heavily penalized. The scoring environment therefore rewards both correctness and efficiency.

2.2.2 State Representation

A key design decision in the implemented search algorithms is that a visited state is not represented only by the agent position. The enemy position changes with time, and target collection changes future goals. The implemented state key therefore includes:

$$state\_key = (agent\_position,\ remaining\_targets,\ enemy\_cycle\_step) \tag{1}$$

This prevents a common bug in dynamic environments: pruning a position as visited even though it occurs at a different time step with a different enemy location.

2.2.3 BFS

`BFS_main.py` implements breadth-first search with a FIFO queue. It is suitable when edge costs are uniform and the primary objective is minimum number of actions. The implementation stores a path beside each frontier state and returns the path when `is_goal_state()` is reached. Collision states are skipped before insertion into the frontier.

A second version, `bfs_with_wait`, introduces an explicit wait-like behavior in blocked enemy situations. It compares normal successors with successors that include wall-directed actions and uses `is_enemy_blocking()` to decide whether waiting is necessary to avoid an immediate trap.

2.2.4 UCS

`UCS_main.py` uses a priority queue ordered by cumulative path cost $g(n)$. This is necessary because bush and grass movement costs are different. UCS expands the cheapest frontier node first and stores the best known cost for each state key:

$$g(n) = \sum_{i=1}^{k} cost(a_i) \tag{2}$$

The implementation uses `heapq`, a monotonically increasing counter for tie-breaking, and a dictionary of visited costs to avoid discarding a cheaper path that reaches the same logical state later.

2.2.5 DLS and IDS

`DLS_main.py` implements recursive depth-limited search and wraps it inside iterative deepening search. Depth-limited search is memory efficient, but it can miss solutions deeper than the chosen limit. IDS solves that by increasing the limit gradually:

$$limit = 0, 1, 2, \ldots, max\_depth \tag{3}$$

The implementation includes both standard and wait-aware variants. The wait-aware version mirrors the BFS logic and adds staying behavior only when the enemy blocks safe progress.

2.2.6 A*

`AStar_main.py` implements an informed search strategy. The heuristic is built from:

- Manhattan distance from the agent to targets,
- an obstacle-aware weighted distance,
- a minimum spanning tree style estimate over remaining targets,
- and a penalty for proximity to enemies.

The heuristic structure can be summarized as:

$$f(n) = g(n) + h(n) \tag{4}$$

where $h(n)$ estimates the cost of collecting remaining targets. The implementation decomposes the problem by searching to the next target, updating the current state, and continuing until all targets are collected. This greedy decomposition keeps the search tractable on harder maps while still benefiting from the heuristic.

2.3 Part B: Linear Regression with SGD

2.3.1 Task

The regression notebook, `linear_regression/modeling_hint.ipynb`, predicts asteroid diameter using orbital and physical features. The original assignment required preprocessing, exploratory data analysis, and a custom implementation of stochastic gradient descent rather than simply calling a ready-made scikit-learn regressor.

2.3.2 Data Pipeline

The notebook uses:

- `pandas` for tabular loading and cleaning,
- `numpy` for vectorized numerical operations,
- `matplotlib` and `seaborn` for visualization,
- `StandardScaler` and `LabelEncoder` for preprocessing,
- train/validation/test splits for evaluation and early stopping.

The final feature matrix contains numeric and encoded categorical features. Scaling is important because SGD is sensitive to feature magnitude; unscaled features would make gradient steps unstable or force a much smaller learning rate.

2.3.3 Model

The implemented model is a scratch linear regressor. For input vector x_i, prediction is:

$$\hat{y}_i = w^T x_i + b \tag{5}$$

The training objective is mean squared error:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{6}$$

The weight update follows stochastic gradient descent with optional improvements such as momentum, learning-rate decay, and early stopping:

$$w \leftarrow w - \alpha \nabla_w MSE \tag{7}$$

2.3.4 Evaluation

The notebook evaluates the trained model using R^2, MAE, and MSE. The original report records a final training loss around 0.001069, and the prediction plots show R^2 values around 0.927, which passes the assignment target of $R^2 \geq 0.8$.

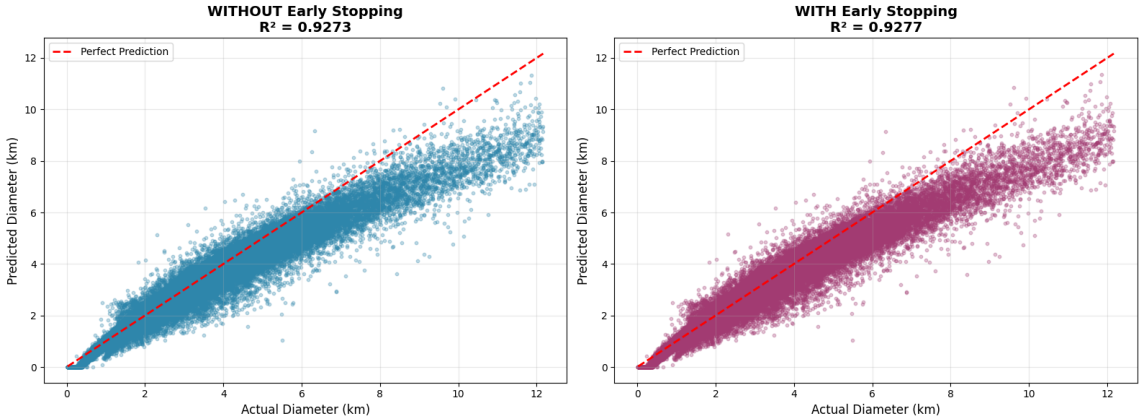

Figure 1: Comparison of regression predictions with and without early stopping.

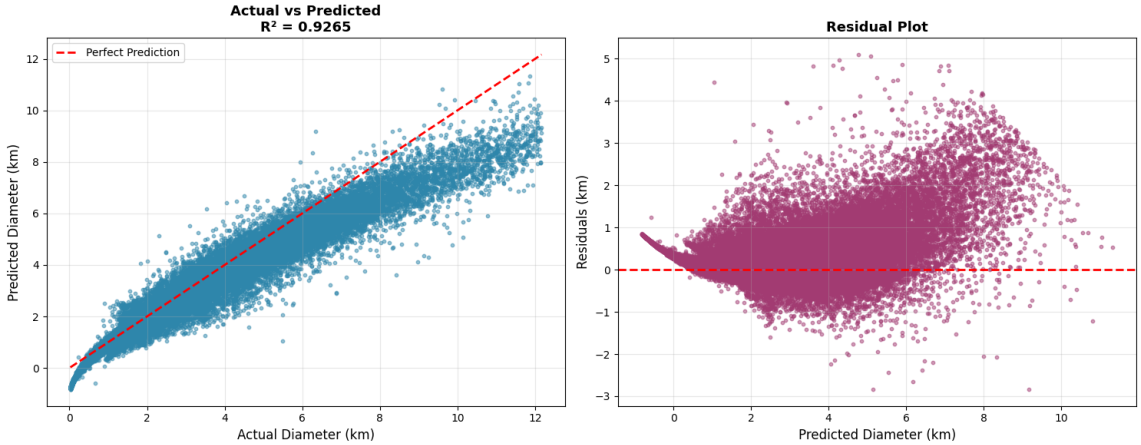

Figure 2: Actual-vs-predicted and residual analysis for the asteroid diameter model.

2.4 Part C: DNA Center Finding

2.4.1 Problem Definition

The DNA task is a closest string problem. Given a set of DNA strings with equal length, the goal is to find a center string minimizing the largest Hamming distance to the input set:

$$\min_{s \in \{A,C,G,T\}^L} \max_{x \in X} d_H(s, x) \tag{8}$$

The evaluation function is therefore:

$$f(s) = \max_{x \in X} d_H(s, x) \tag{9}$$

2.4.2 Implemented Local Search Methods

The notebook `simulated_annealing/DNA_Center.ipynb` implements the required functions for state initialization, evaluation, neighbor generation, and local search. The implementation compares:

- hill climbing,
- random-restart hill climbing,
- stochastic hill climbing,
- random-walk hill climbing,
- simulated annealing,
- and brute force for small instances.

Hill climbing always moves to a better neighbor and can become stuck in local minima. Simulated annealing can temporarily accept worse states according to:

$$P(accept) = e^{-\Delta/T} \tag{10}$$

where Δ is the increase in cost and T is the temperature. This makes annealing more robust on rugged search spaces.

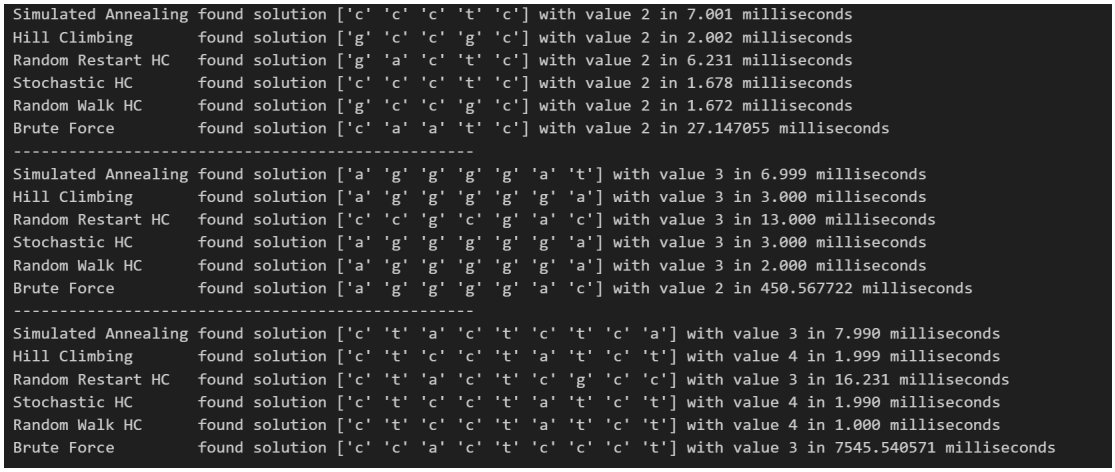

Figure 3: Extracted comparison output for DNA center search methods.

3 Phase 2: MDP and Reinforcement Learning

3.1 Scope

Phase 2 studies a stochastic WALL-E environment. The phase is split into a known-environment MDP solver, an unknown-environment Q-learning solver, and a DQN bonus.

3.2 Environment

The main grid is 8×8. WALL-E starts at the upper-left corner and must reach the plant while collecting trash, avoiding buildings and enemies, and respecting a maximum number of actions. Actions are stochastic: when the agent selects a direction, the actual movement may follow the intended direction or one of its neighboring directions according to a fixed transition distribution created for the environment.

Important rewards and penalties include:

- goal reward: $+400$,
- default movement reward: -1,
- enemy collision: -400,
- trash collection reward: $+250$ in the fully collectable case,
- limited action budget, usually 150 moves.

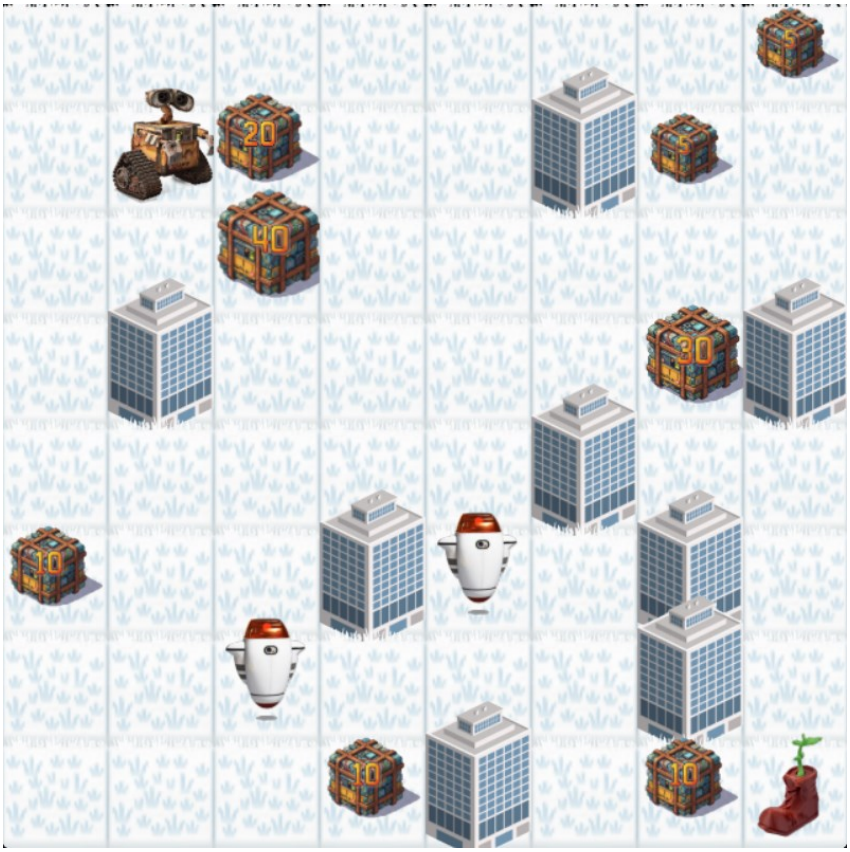

Figure 4: Known-environment WALL-E grid with buildings, trash, enemies, and goal.

3.3 Known Environment: Value Iteration

3.3.1 Transition Model

The known MDP solver uses the transition model exposed by the environment. For each state-action pair, the model enumerates possible next states, probabilities, and rewards. The value iteration update is:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V_k(s') \right] \tag{11}$$

The implemented solver uses $\gamma = 0.99$, a small convergence threshold, and up to 5000 iterations.

3.3.2 Reward Maps

The project does not rely on a single static reward map. Instead, `mdp/main.py` builds multiple reward maps:

- `build_reward_map_for_goal`: emphasizes reaching the plant safely,
- `build_reward_map_for_trash`: targets a specific trash item,
- `build_reward_map_for_collection`: encourages general collection based on current power threshold.

The policy therefore depends on a meta-state, especially the current power threshold. A trash item with value 40 should not be treated the same when WALL-E has power 5 and when it has power 40.

3.3.3 Policy Selection

After precomputing policies, the execution loop uses `select_optimal_policy`. This policy selector considers:

- remaining moves,
- distance to the goal,
- accessible trash,
- risk around enemies,
- current power threshold,
- stuck detection based on recent position history.

This architecture separates expensive planning from real-time execution. Value iteration runs before play, while the policy selector picks the relevant precomputed policy during play.

3.3.4 Visualization

The solver saves heat maps and convergence curves for goal, trash, and collection policies. This was part of the phase requirement and helps inspect whether value iteration is converging and whether the learned value surface makes sense.

3.4 Unknown Environment: Q-Learning

3.4.1 Problem Changes

In the unknown environment, the agent does not know the reward or transition model in advance. It must learn from interaction. The state is richer than position alone:

$$s = (position,\ power\_level,\ trash\_mask) \tag{12}$$

The `trash_mask` compresses remaining trash information into a bitmask. The implementation lazily initializes Q-values only when a state is encountered.

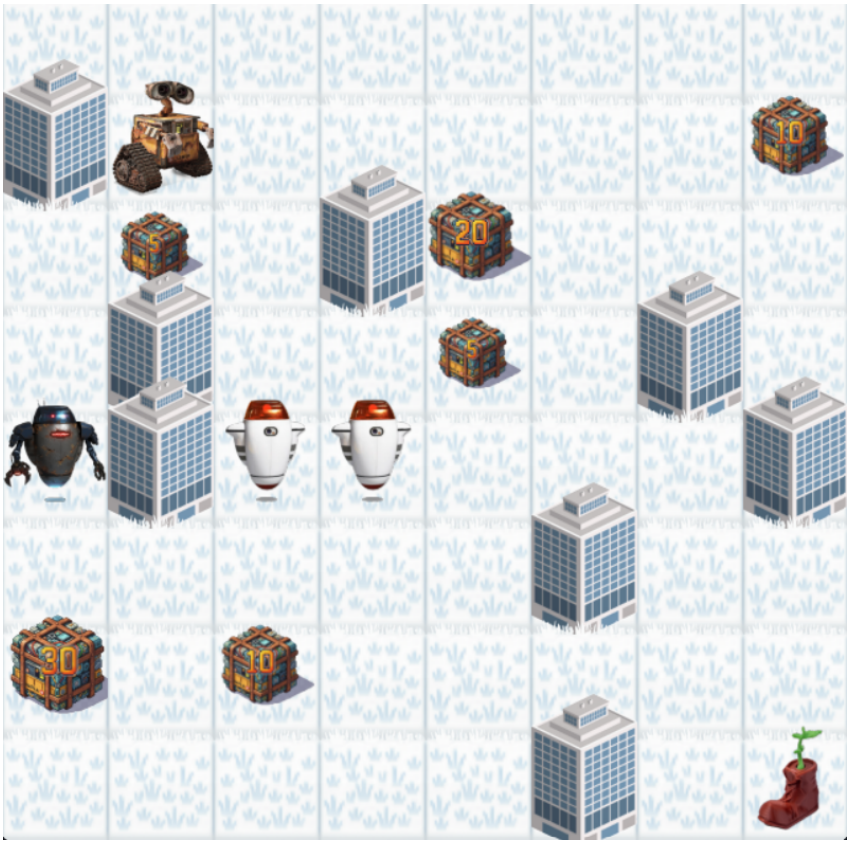

Figure 5: Unknown-environment variant with hunter, enemies, buildings, trash, and goal.

3.4.2 Q-Learning Update

The agent applies the standard Q-learning update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{13}$$

The implemented parameters include learning-rate decay, epsilon decay, a minimum epsilon, and a minimum learning rate. Training is configured for 30,000 episodes. Every 1000 episodes, the code records a value-difference metric by comparing the current and previous Q-tables.

3.4.3 Evaluation

After training, the script tests the greedy policy, saves a convergence plot, generates a policy map for a reference state, and writes a summary file. The grading thresholds from the phase document focus on mean score, with higher scores corresponding to stronger learned behavior.

3.5 Bonus: Dueling DQN

The DQN bonus uses PyTorch to approximate Q-values with a neural network. The implemented architecture is dueling:

$$Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s,a') \tag{14}$$

The agent includes:

- a policy network and target network,

- replay buffer,
- mini-batch training,
- target-network synchronization,
- epsilon-greedy action selection,
- vector state representation with position, power, and trash state.

The bonus environment is smaller and simplified compared with the full 8×8 environment, which makes neural training more practical.

4 Phase 3: Adversarial Search Game

4.1 Environment

The PaintBattle project implements a competitive two-player grid game. Each player moves across a map and paints territory. Capturing enclosed regions can change the score quickly, so a locally good move may be bad after the opponent responds. This makes the project a natural adversarial search problem.

The main environment is `src/env.py`. It defines:

- `PBState`: immutable-style state container for grid, paint matrix, player positions, step count, and alive flags,
- `get_successors(team)`: valid successor generation for a team,
- `capture_logic`: local region capture behavior,
- `get_scores`: territory scoring,
- `PaintBattle.play`: full Pygame loop for two agents.

4.2 Agent Interface

Every player inherits from the base `Agent` class and implements:

```
def get_action(self, state):
    raise NotImplementedError
```

The final AI agent is `MinimaxAgent` in `Minimax1.py`. In `main.py`, it is configured with depth 9 and played against packaged binary enemy agents.

4.3 Map Example

```
GGGGGGGGG1
GGGGGGGGGG
GGGGGGGGGG
GGGGGGGGGG
GGGGGGGGGG
GGGGGGGGGG
GGGGGGGGGG
GGGGGGGGGG
GGGGGGGGGG
2GGGGGGGGG
```

Here, `G` represents normal ground and `1/2` mark player start positions. Other maps introduce blocked regions with `R`.

4.4 Minimax with Alpha-Beta Pruning

The minimax tree alternates between maximizing the controlled agent's utility and minimizing it under the opponent's best response. The recursive value is:

$$minimax(s,d) = \begin{cases} eval(s), & d = 0 \text{ or terminal}(s) \\ \max_{s' \in Succ(s)} minimax(s', d-1), & \text{max turn} \\ \min_{s' \in Succ(s)} minimax(s', d-1), & \text{min turn} \end{cases} \tag{15}$$

Alpha-beta pruning avoids branches that cannot affect the final decision:

$$\alpha = \max(\alpha, v), \quad \beta = \min(\beta, v), \quad \alpha \geq \beta \Rightarrow prune \tag{16}$$

4.5 Heuristic Evaluation

The game is usually not searched until a terminal state, so evaluation quality matters. The implemented `evaluate` function combines:

- score difference between teams,
- alive/dead status,
- mobility count,
- territory control,
- distance pressure,
- opponent constraints,
- and local tactical opportunities.

The agent also uses quick move evaluation and move ordering. Better ordering improves alpha-beta effectiveness because good moves raise α or lower β earlier.

5 Phase 4: First-Order Logic Minesweeper

5.1 Problem

Minesweeper is a partial-information puzzle. Revealed cells provide numeric clues, hidden cells may contain mines, and the solver must infer safe moves without directly observing mine locations. The phase document required using first-order logic style reasoning through PyDatalog or a Prolog-like engine.

5.2 Environment

The environment is implemented in `src/minesweeper.py`. It provides:

- `reveal(r,c)` returning -1 for a mine or a clue from 0 to 8,
- `flag(r,c)` and `unflag(r,c)`,
- deterministic board generation through a seed,
- Pygame rendering and HUD output,
- victory detection and mine counting.

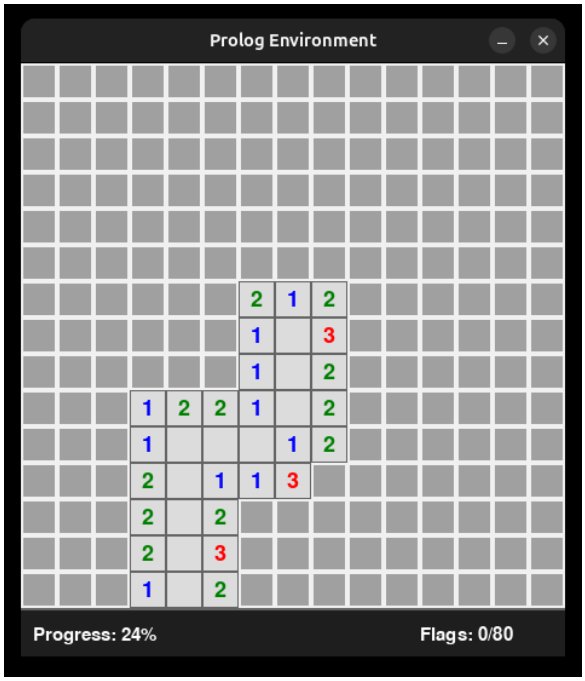

Figure 6: Extracted solver-state visualization from the Minesweeper phase documentation.

5.3 Knowledge Representation

The solver in `main.py` defines PyDatalog terms:

- `hidden(R,C)`
- `revealed(R,C)`
- `flagged(R,C)`
- `neighbor(R,C,NR,NC)`
- `clue(R,C,V)`
- `flagged_count(R,C,FC)`
- `hidden_count(R,C,HC)`
- `safe(R,C)`, `mine(R,C)`, `safe_zero(R,C)`, `frontier(R,C)`

Board topology is asserted once at startup. Dynamic facts are updated after each reveal or flag.

5.4 Inference Rules

The main safe-cell rule is:

$$clue(R, C, V) \wedge flagged\_count(R, C, V) \Rightarrow hidden\ neighbors\ are\ safe \tag{17}$$

The main mine rule is:

$$flagged\_count(R, C, FC) + hidden\_count(R, C, HC) = V \Rightarrow hidden\ neighbors\ are\ mines \tag{18}$$

The solver also has a zero-clue rule:

$$clue(R, C, 0) \Rightarrow all\ hidden\ neighbors\ are\ safe \tag{19}$$

To reduce unnecessary queries, the solver uses a `frontier` predicate: only revealed cells adjacent to at least one hidden cell need to be considered.

5.5 Control Loop

Each cycle performs:

1. render the board,
2. compute temporary flagged/hidden neighbor counts,
3. query `safe_zero`, then `safe`, then `mine`,
4. batch reveal or flag actions,
5. retract temporary count facts,
6. fall back to risk-based guessing if no logical move exists.

The fallback is not pure FOL, but it is necessary because Minesweeper can reach states where deterministic local inference is insufficient.

6 Phase 5: LLM + FOL Mastermind

6.1 Problem

Mastermind is a code-breaking game. A secret code is generated from an alphabet, and each guess receives three feedback values:

- correct: right symbol in the right position,
- misplaced: right symbol in the wrong position,
- incorrect: symbol not present in the secret.

Figure 7: Mastermind board image extracted from the original bonus documentation.

6.2 Manual Game

`manual_game.py` provides a direct interactive version. The default configuration is:

- alphabet: `ABCDE`,
- code length: 5,
- maximum turns: 12.

This file is useful for validating the environment independently of LLM and Prolog dependencies.

6.3 Logic-LM Architecture

The bonus document is based on the Logic-LM pattern: natural language models generate symbolic formulations, and a symbolic reasoner checks or solves them. In this project, the LLM is not trusted blindly. Its output is parsed and validated before it becomes part of the Prolog knowledge base.

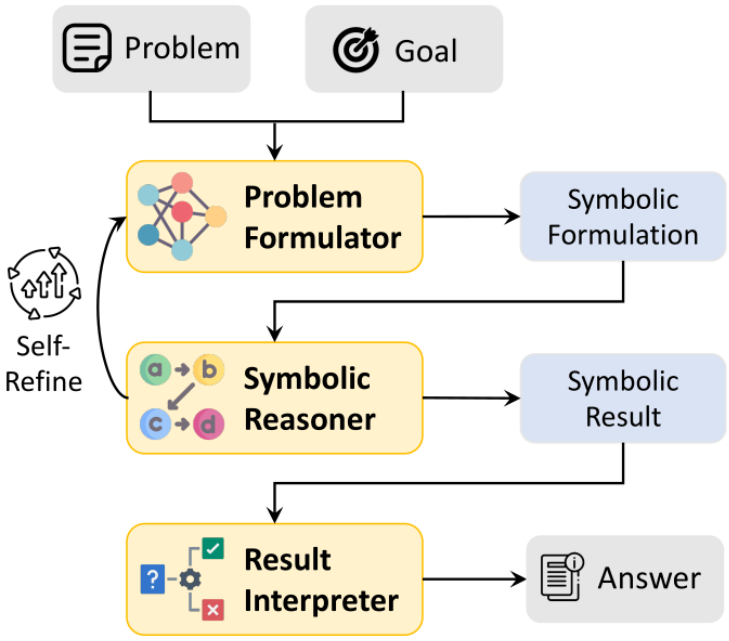

Figure 8: Logic-LM architecture from the original bonus phase documentation.

6.4 Generated Prolog Base

`llm_solver.py` writes `logic.pl` dynamically for the selected difficulty. It defines:

- the valid alphabet,
- code length,
- valid-code predicate,
- `calc_score/5`,
- correct-position counting,
- removal of exact matches,
- misplaced counting through symbol occurrences.

The core scoring relation is:

```
calc_score(Secret, Guess, C, M, I) :-
    calc_correct(Secret, Guess, C),
    remove_correct(Secret, Guess, S_Rest, G_Rest),
    calc_misplaced(S_Rest, G_Rest, M),
    code_length(Len),
    I is Len - C - M.
```

6.5 LLM Constraint Generation

After each guess, the solver asks the LLM to generate a rule like:

```
turn_1(Candidate) :-
    calc_score(Candidate, ['A','B','C','D','E'], 1, 2, 2).
```

The implementation validates:

- that the rule matches the expected predicate structure,
- that the feedback values sum to the code length,
- that no feedback value is negative,
- that the LLM did not alter the actual feedback values.

If validation fails, the code uses a deterministic fallback rule generated by Python. This is an important engineering choice: the LLM helps formulate constraints, but the solver remains robust against hallucinated or malformed output.

6.6 Candidate Selection

The `PrologSolver` asserts validated constraints and queries:

```
valid_code(C), turn_1(C), turn_2(C), ...
```

The resulting candidates are passed back to the LLM for strategic selection when there are multiple possibilities. If the LLM returns an invalid candidate, the code retries and eventually falls back to a random valid candidate. The final implementation also removes hardcoded credentials and reads:

- `OPENAI_API_KEY`,
- `OPENAI_BASE_URL`,
- `OPENAI_MODEL`.

7 Repository Cleanup and Engineering Decisions

7.1 Files Preserved

The final repository preserves:

- source code for every phase,
- notebooks for regression and DNA center search,
- maps and icons required by Pygame environments,
- binary enemy agents required by the PaintBattle environment,
- LaTeX source and rendered PDF documentation,
- phase-level README files.

7.2 Files Removed or Ignored

The cleanup excludes:

- nested `.git` directories from source clones,
- Python cache directories,
- generated training plots and screenshots,

- temporary `logic.pl`,
- LaTeX auxiliary files,
- hardcoded API credentials.

7.3 Verification

The consolidated repository was checked with:

- Python syntax compilation with `python -m compileall projects`,
- notebook JSON validation,
- secret scanning for the previously hardcoded API key pattern,
- two-pass LaTeX rendering with `pdflatex`,
- Git status verification after commit and push.

8 Execution Guide

8.1 Install Dependencies

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

8.2 Run Phase 1 Search

```
cd projects/phase-01-search-regression-annealing/search_algorithms
python BFS_main.py
python UCS_main.py
python DLS_main.py
python AStar_main.py
```

8.3 Run Phase 2

```
cd projects/phase-02-mdp-reinforcement-learning/mdp
python main.py

cd ../unknown_environment
python main.py

cd ../dqn_bonus
python main.py
```

8.4 Run Phase 3

```
cd projects/phase-03-adversarial-search-game
python main.py
```

8.5 Run Phase 4

```
cd projects/phase-04-fol-minesweeper
python main.py
```

8.6 Run Phase 5

```
cd projects/phase-05-llm-fol-mastermind
python manual_game.py

set OPENAI_API_KEY=your_api_key_here
set OPENAI_BASE_URL=https://api.openai.com/v1
set OPENAI_MODEL=gpt-4o-mini
python llm_solver.py
```

9 Conclusion

This portfolio demonstrates a broad progression across AI methods. Phase 1 starts with deterministic graph search, regression, and local optimization. Phase 2 introduces stochastic transitions, value iteration, tabular reinforcement learning, and neural Q-value approximation. Phase 3 adds competitive reasoning with minimax and alpha-beta pruning. Phase 4 formalizes game knowledge as logical facts and rules. Phase 5 connects modern LLM interfaces to symbolic reasoning through validated Prolog constraints.

The final repository is designed for submission, review, and future extension. It keeps the original implementations intact where they are needed, documents the reasoning behind the algorithms, and provides a clean structure for navigating the complete AI fundamentals project set.